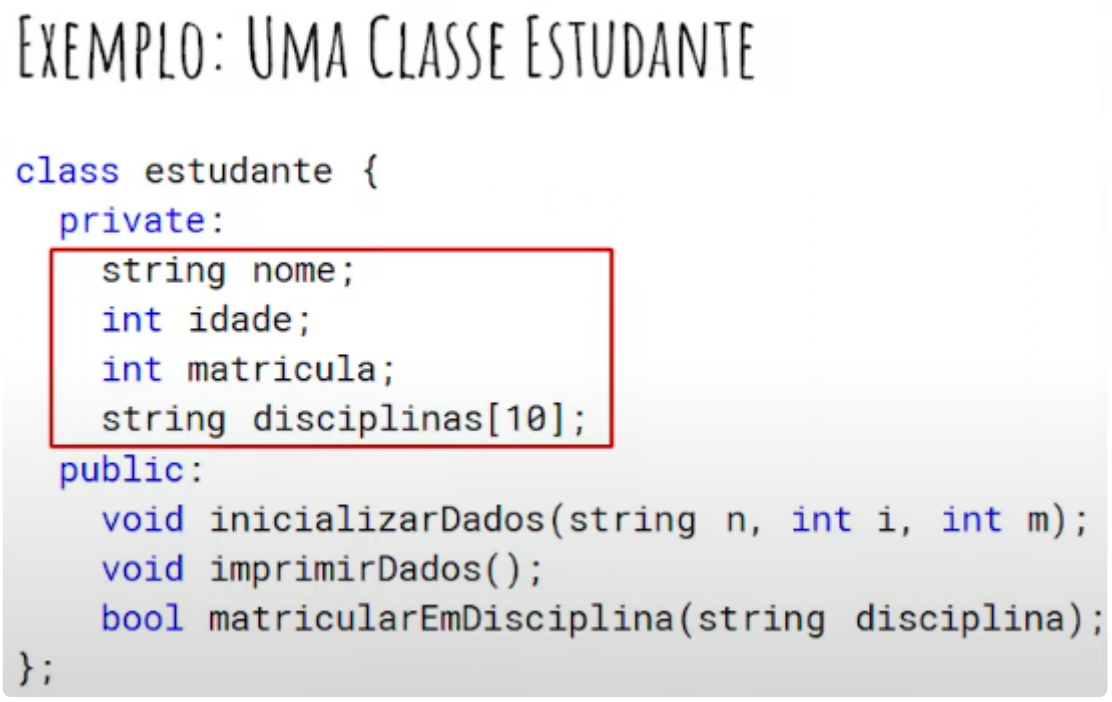
Revisão de POO

Revisitaremos dois pilares importantíssimos da programação Orientada a Objetos para esse curso.

Abstração

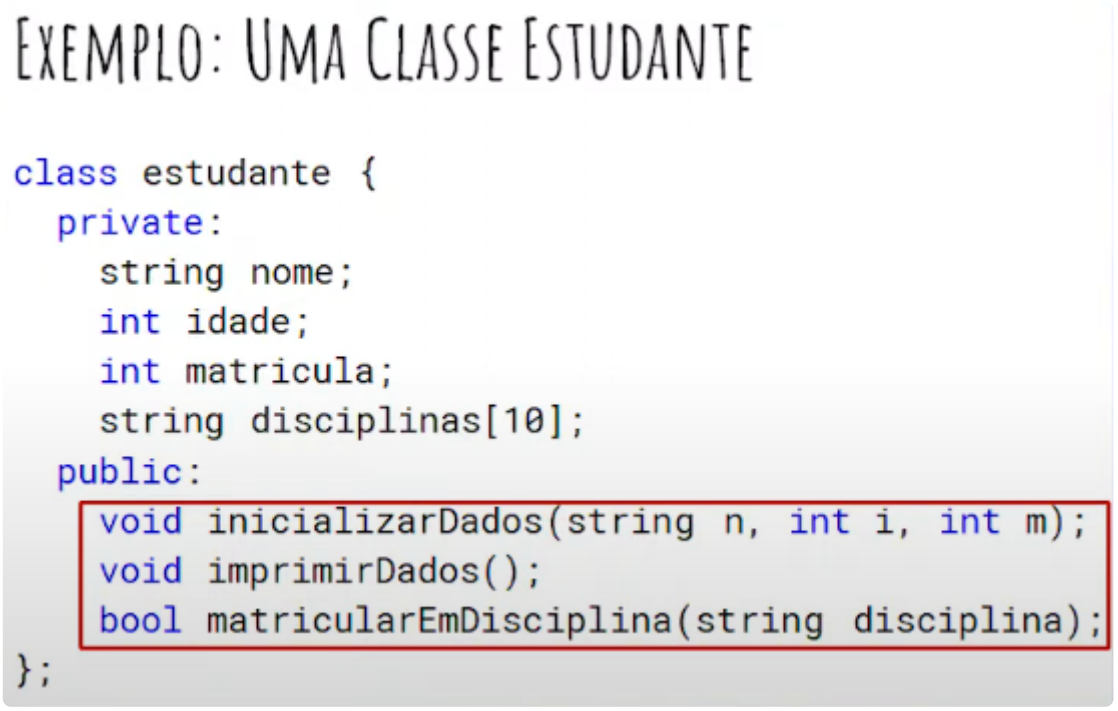
Representação de uma entidade do mundo real, com seu comportamento e características. Para fazermos uso da abstração, utilizamos:

- Objetos
  É um conceito do mundo real aplicável ao mundo computacional. Cada objeto é uma instância única de uma classe (responsável por definir as diretrizes gerais de funcionamento desse objeto).
- Classes
  Tipo abstrato de dados ([TAD](Tipos Abstratos de Dados)) que reúne objetos com características semelhantes. Em C++, `struct`'s são classes com todos os seus atributos e métodos públicos por padrão.
- Atributos
  Ditam as características de objetos reunidos em uma classe. Para a classe `Estudante`, por exemplo, podemos lidar com os atributos `idade`, `altura`, `nome`, `matricula`, etc. Para além de tipos simples como *int*, *string* e *float*, podemos criar atributos que utilizam-se de objetos de outras classes (exemplo: classe `Disciplina` com um atributo chamado `alunos` do tipo `Estudante`).



- Métodos
  Ditam os comportamentos de objetos reunidos em uma classe. Para a classe

`Professor`, por exemplo, podemos lidar com os atributos `passarAvaliacao()`, `corrigirExercicios()`, `ministrarAula()`, etc.



No exemplo acima, `inicializarDados()` pode ser considerado como um exemplo de construtor (apesar de não seguir a sintaxe exata de um).

Encapsulamento

As classes possuem como característica o controle de visibilidade, que pode ser:

- Público
  Qualquer elemento do sistema tenha acesso aos atributos/métodos dessa classe.
- Privado
  Somente elementos dessa mesma classe têm acesso aos seus atributos/métodos.
- Protegido
  Elementos dessa mesma classe (ou classes filhas) têm acesso aos seus atributos/métodos.

A função geral do encapsulamento é a ocultação de dados para melhor funcionamento do software, facilitando o entendimento, reuso e manutenção do programa. Além disso, ele provê segurança, independência entre classes, aumenta o grau de abstração para o leitor do código e minimiza o acoplamento entre elas.
Em C++, podemos romper esse encapsulamento com o uso da diretiva `friend` portanto, devemos usá-la com cuidado.

Mais conceitos de POO...

Construtores e Destrutores

Um **construtor** é um método especial que **inicializa um objeto para um estado adequado no momento da sua criação**. Ele garante uma configuração correta de seus atributos (sendo eles tipos simples ou mesmo outros objetos).

Já um **destrutor** é um método destinado a liberar a memória ocupada por um determinado objeto, sendo geralmente chamados quando termina esse chega ao fim de seu tempo de vida. Em algumas linguagens, como C++, são essenciais devido à necessidade de desalocação de memória (por não terem um *garbage collector*).
O destrutor, diferentemente do construtor, é precedido por um `~`.

```
class Foo {

    ...

public:
    Foo() {
      n = 0;
      c = 'a';
      f = 0;
      s = "a";
      vet = new int[10];
    }

    ~Foo() {
      delete[] vet;
    }

    ...

};
```

Inicialização de atributos em construtores

Atributos em classes são inicializados geralmente de duas maneiras:

- Inicialização direta
  Na inicialização direta, os valores dos atributos são definidos diretamente no corpo do construtor.
- Inicialização em lista (repasse ao construtor do atributo)
  Na inicialização em lista, os valores dos atributos são definidos diretamente na chamada do construtor. A depender do compilador, essa inicialização pode ser a mais rápida.

```
class myClass
{
  public:
    myClass(float f, char a, bool b, int i);

  private:
    float mFloat;
    char mCharacter;
    bool mBoolean;
    int mInteger;
};

myClass::myClass(float f, char a,
    bool b, int i) :
      // Inicialização em lista
      mFloat( f ), mBoolean( true )
{
    // inicialização direta
    mCharacter = a;
    mInteger = 0;
}
```

OBS:

- Atributos constantes ou referências só podem ser inicializados em listas.
- Vetores ou EDs sem construtores só podem ser inicializados diretamente.

Sobrecarga de métodos

Em C++, podemos ter vários métodos com o mesmo nome, desde que sua assinatura (isto é, seus **parâmetros**) sejam diferentes umas das outras.

```
// Exemplos de sobrecarga:
int foo(int x);
float foo(float f);
int foo(char c);
float foo(float f, char c, int b);
```

Construtores também podem ser sobrecarregados, havendo diversas maneiras de se inicializar um objeto.